



北京大学

本科生毕业论文

题目: **Householder 变换矩阵稳定性
定性的 Lean 形式化证明**

**Formalization of the Stability
Proof for the Householder
Transformation Matrix in Lean**

姓名: 蒋穆清

学号: 2200010766

院系: 数学科学学院

专业: 信息科学方向

导师姓名: 夏壁灿

二〇二六年五月

版权声明

任何收存和保管本论文各种版本的单位和个人，未经本论文作者同意，不得将本论文转借他人，亦不得随意复制、抄录、拍照或以任何方式传播。否则，引起有碍作者著作权之问题，将可能承担法律责任。

摘要

QR 分解是数值线性代数中的基础问题，在科学计算与工程应用中具有广泛用途。Householder 变换作为求解 QR 分解的重要方法，其数值稳定性直接影响算法的可靠性。随着形式化验证技术的发展，利用定理证明系统对数值算法进行严格验证，已成为提高计算结果可信性的重要研究方向。本文以 Householder 变换的稳定性分析为对象，基于 Lean 定理证明系统，对相关数学结论进行了形式化证明。论文首先对浮点误差模型以及 Householder 变换中的关键计算过程进行了形式化描述，在此基础上完成了稳定性定理的证明，并给出了误差界的具体常数。研究表明，在合理的机器精度条件下，计算误差能够随向量维度保持线性增长，从而保证算法具有良好的数值稳定性。本文工作展示了 Lean 在数值线性代数算法验证中的应用能力，为后续数值算法的可信形式化研究提供了参考。

关键词：数学形式化，Lean

ABSTRACT

QR decomposition is a fundamental problem in numerical linear algebra with wide applications in scientific computing and engineering. As a key method for computing QR decompositions, Householder transformations play an important role in ensuring numerical stability. With the development of formal verification, Lean has become an effective tool for rigorously analyzing and certifying numerical algorithms. This paper presents the formalization of the stability of Householder transformations using the Lean prover. This project formalized the floatpoint error framework and the essential computational steps involved in Householder transformations, and established a formal proof within Lean. Based on this formalization, explicit error bounds are derived, showing that under reasonable machine precision assumptions, the computational error grows linearly with the vector length. This result confirms the numerical stability of the algorithm and demonstrates the effectiveness of Lean in the formal verification of numerical linear algebra proofs, providing a foundation for further work on trustworthy and machine-checked numerical analysis.

KEYWORDS: Formalization, Lean

目 录

第一章 引言	1
1.1 研究背景	1
1.2 研究内容	1
第二章 理论基础	2
2.1 Householder 矩阵	2
2.1.1 定义与性质	2
2.1.2 Householder 变换	2
2.2 QR 分解	2
2.2.1 定义	3
2.2.2 Householder 矩阵在求解 QR 分解问题中的应用	3
2.3 浮点计算	3
2.3.1 浮点数存储	4
2.3.2 舍入误差	4
2.3.3 浮点计算误差模型	4
第三章 Householder 法的误差估计	6
3.1 记号	6
3.2 基础浮点运算的误差估计	6
3.3 Householder 过程	11
3.4 Householder 法的误差估计	13
参考文献	16
附录	17
致谢	18

第一章 引言

1.1 研究背景

数值线性代数是科学计算的重要基础，在机器学习、计算机图形学、信号处理、工程仿真以及高性能计算等领域具有广泛应用。其中，QR 分解作为数值线性代数中的核心工具，为线性方程组求解、最小二乘问题、特征值计算等任务提供了统一而高效的计算框架。

Householder 变换是实现 QR 分解的重要方法。与经典 Gram-Schmidt 正交化相比，Householder 方法能够有效减小浮点舍入误差带来的影响，因此在实际计算中被广泛采用。其核心思想是通过构造正交反射矩阵，将向量逐步变换为目标方向，从而实现矩阵的正交化过程。由于 Householder 方法涉及大量浮点运算，其数值稳定性分析一直是数值分析领域的重要研究内容。

传统数值分析通常通过数学推导对算法误差进行估计，并在一定假设条件下证明算法的稳定性。然而，这类证明往往包含复杂的符号推演与多层次误差传播分析，人工验证过程容易出现疏漏。因此，如何利用形式化方法对数值算法进行机器可验证的严格证明，逐渐成为形式化数学与可信计算研究中的重要方向。

近年来，定理证明辅助系统的发展为数学证明的形式化提供了有效工具。其中，Lean 作为一种基于依赖类型理论的交互式定理证明系统，兼具较强的表达能力与较高的自动化水平，已被广泛应用于数学定理验证、程序正确性证明以及形式化数学库构建等领域。通过 Lean 对数值算法的稳定性结论进行形式化，不仅能够提高证明过程的可靠性，也有助于推动可信科学计算的发展。

1.2 研究内容

本文围绕 Householder 变换的数值稳定性问题展开研究，并基于 Lean 定理证明系统对相关结论进行形式化验证。研究的核心问题是：在浮点误差模型下，Householder 变换中关键向量的计算结果是否能够保持足够小的相对误差，以及如何以形式化方式严格证明该性质。

为此，本文首先对 Householder 变换的数学定义及其稳定性理论进行了分析，并建立相应的浮点误差模型。在此基础上，使用 Lean 对相关数学对象、误差界以及稳定性定理进行形式化表示与证明。本文进一步给出了误差界中的具体常数，从而使稳定性结论具有更明确的量化形式。这表明形式化方法不仅能够验证已有理论结果，还能够帮助将传统证明中被省略的步骤补全。

第二章 理论基础

本章与下一章中都将使用参考证明的源文献 *Accuracy and Stability of Numerical Algorithms* 中的记号。

2.1 Householder 矩阵

Householder 矩阵是数值线性代数中一种重要的正交变换工具，广泛应用于 QR 分解等矩阵计算算法中。其核心作用是通过一种对称的“反射”变换，将向量或矩阵逐步转换为更便于计算的形式。相比 Gram-Schmidt 正交化等其他正交化方法，Householder 法具有更好的数值稳定性，因此在现代科学计算软件与高性能数值计算中被广泛采用。

2.1.1 定义与性质

Householder 矩阵（或称 Householder 变换矩阵）是具有如下形式的矩阵：

$$P = I - \frac{2}{v^T v} v v^T, \quad 0 \neq v \in \mathbb{R}^n$$

其对称性显然。又由

$$\begin{aligned} P^2 &= I - \frac{4}{v^T v} v v^T + \frac{4}{(v^T v)^2} v v^T v v^T \\ &= I - \frac{4}{v^T v} v v^T + \frac{4}{(v^T v)^2} v (v^T v) v^T = I \end{aligned}$$

结合其对称性可知其为正交矩阵。

2.1.2 Householder 变换

考虑方程 $Px = y$ ，其中 $\|x\|_2 = \|y\|_2$ 。我们令 P 为一个 Householder 矩阵。方程变为：

$$x - 2\left(\frac{v^T x}{v^T v}\right)v = y$$

令 $v = x - y$ ，我们有

$$v^T v = x^T x + y^T y - 2x^T y$$

又有 $\|x\|_2 = \|y\|_2$ ，即 $x^T x = y^T y$ 。因此

$$v^T x = x^T x - x^T y = \frac{1}{2} v^T v$$

从而 $v = x - y$ 且 $P = I - \frac{2}{v^T v} v v^T$ 是方程的一个解。注意 $x = y$ 时我们需要特别定义 $P = I$ 防止出现除 0 的情况。

2.2 QR 分解

QR 分解是数值线性代数中的一种重要矩阵分解方法，在科学计算、数据分析、机器学习以及工程计算等领域具有广泛应用。通过 QR 分解，可以将一个复杂矩阵转化为更易处理的

形式,从而便于求解线性方程组、最小二乘问题以及特征值问题等计算任务。QR 分解在实际任务中非常重要,是现代数值算法中的基础工具之一。

2.2.1 定义

熟知对矩阵 $A \in \mathbb{R}^{m \times n}$ 且 $m \geq n$, 总存在如下的分解:

$$A = QR = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = Q_1 R_1$$

其中 $Q \in \mathbb{R}^{m \times m}$ 为正交矩阵, $R_1 \in \mathbb{R}^{n \times n}$ 为上三角矩阵。

2.2.2 Householder 矩阵在求解 QR 分解问题中的应用

下面以一 4×3 矩阵为例, 展示 Householder 法求解 QR 分解问题的过程。

$$A = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{P_1} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{P_2} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{P_3} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \end{bmatrix} = R$$

一般地, 我们考虑以下递归过程。记 $A_1 = A$ 。对递归的第 k 步, 设我们面对如下矩阵:

$$A_k = \begin{bmatrix} R_{k-1} & z_k & B_k \\ 0 & x_k & C_k \end{bmatrix}, \quad R_{k-1} \in \mathbb{R}^{(k-1) \times (k-1)}, x_k \in \mathbb{R}^{n-k+1}$$

其中 R_{k-1} 为上三角矩阵。那么令

$$P_k = \begin{bmatrix} I_{k-1} & 0 \\ 0 & \widetilde{P}_k \end{bmatrix}$$

其中 $\widetilde{P}_k$ 为求解 $\widetilde{P}_k x_k = \sigma e_1$ 得到的 Householder 矩阵。这里 σ 可以取为任意值, 只需保证 $x_k \neq e_1$ 。

由 2.1.2 中提到的性质, 我们知道不断重复次过程可以得到满足要求的 QR 分解。

注意在实际应用中, 我们不会显式的将 P_k 计算出来, 而是直接使用 $\widetilde{P}_k C_k = C_k - \frac{2}{v^T v} v(v^T C_k)$ 计算下一步递归。因此我们只需关注 v 的计算精度。

2.3 浮点计算

在现代计算机系统中, 实数运算无疑在各类科学计算中都扮演重要的角色。然而, 由于计算机硬件只能以有限位二进制形式存储数据, 实数在机器中的表示不可避免地具有近似性。实践中计算机通常采用浮点数表示实数。浮点表示方法借鉴了科学计数法的思想, 通过尾数

与指数的组合实现对不同数量级实数的统一表示，从而成为现代计算机数值计算的核心基础。

尽管浮点表示极大提高了计算机处理实数的能力，但浮点计算误差问题始终是数值计算研究中的重要内容。由于浮点数只能对真实数值进行有限精度的近似表示，在数据存储、格式转换以及算术运算过程中都会产生误差。这些误差在大量迭代运算中可能不断累积与传播，从而影响最终计算结果的稳定性与可靠性。在高精度科学计算、数值模拟以及机器学习训练等场景中，微小的浮点误差甚至可能导致结果失真、算法不收敛或系统行为异常。因此，分析浮点误差的产生机制及其传播规律，对于提高数值计算的准确性与算法鲁棒性具有重要意义。

2.3.1 浮点数存储

计算机中的浮点数遵循 IEEE 754 标准，其存储结构主要由符号位、指数位和尾数位三部分构成。其中，符号位用于表示数值正负；指数位用于表示数值的数量级；尾数位用于存储有效数字。以单精度浮点数为例，其使用 32 个比特位存储一个实数。大部分浮点数（规格化浮点数）被表示为： $(-1)^s \times (1.m) \times 2^{e-127}$

其中 s, e, m 均为二进制串。它们长共为 32 位，构成了一个单精度浮点数。 s 表示符号位，占据 1 个比特位。 e 表示指数，占据 8 个比特位。 m 表示尾数，占据剩余的 23 个比特位。

2.3.2 舍入误差

分析如上的浮点数存储模型，我们很容易发现一些运算即使在运算数都是浮点数的情况下，得到的结果也无法用同样精度的浮点数存储。遇到这种情况时计算机将其舍入到最近的能被存储的实数并存储，这就造成了浮点数运算中的舍入误差。

我们考虑以下简化的模型中的一次计算。设我们有一个 8 位浮点数存储系统，8 位由 1 位符号位，3 位指数位和 4 位尾数位组成：

$$2^4 \times 1.1110 + 2^3 \times 1.0011 = 30 + 9.5 = 39.5 = 2^5 \times 1.001111$$

注意到这里最终的得数的尾数已经不能存入 4 位的尾数位了，因此强制被舍入：

$$2^5 \times 1.001111 \approx 2^5 \times 1.0100 = 40$$

这就是舍入误差的概念。虽然受限于浮点数存储系统，舍入误差无法避免，但它仍然具有一些性质，使我们我们可以对其进行分析，并控制误差。

2.3.3 浮点计算误差模型

注意到舍入误差只会发生在尾数位不够用的情况下，因此若我们使用 m 位尾数位的存储方法，每次舍入时产生的误差绝对值都不会超过原数字绝对值的 $1/2^{m+1}$ 。由此我们可以采用以下的浮点误差描述模型：

设 $x, y \in \mathbb{R}$ ，记 $op \in \{+, -, \times, /\}$ 为基本浮点运算。本文采用如下经典浮点误差模型描述一次浮点运算的结果：

$$fl(x \text{ op } y) = (x \text{ op } y)(1 + \delta), \quad |\delta| \leq u$$

其中 u 是一个仅与浮点数存储系统有关的小正数。

注意到有一部分操作一定不会导致舍入误差，例如取相反数，或者与 0 相加等等。在后面的证明中会用到这一性质。

该模型的优势在于，它能够将一次浮点运算产生的误差统一表示为一个有界的相对扰动，从而使复杂算法中的误差传播分析可以转化为对多个扰动项的组合估计。相比直接分析底层二进制表示，该模型在保留主要误差特征的同时，大幅降低了理论分析的复杂度。此外，该模型中的误差界仅依赖于机器精度，而与具体运算数值无关，因此分析得到的结果对输入是鲁棒的。在后续证明中，本文将反复利用该模型对连续浮点运算中的误差进行估计，并给出误差的上界。

第三章 Householder 法的误差估计

本章陈述本项目核心内容，证明使用 Householder 法得到的确定 Householder 矩阵的向量 v 的误差关于 n 和 u 的明确上界。

3.1 记号

定义 3.1. 浮点计算精度 u

```
variable (u : NNReal)
```

浮点误差模型部分和上一章中定义相同，不再赘述。

定义 3.2. 浮点误差阶数

```
def γ (n : N) : R := (n : R) * (u : R) / (1 - (n : R) * (u : R))
```

我们定义 $\gamma_n = nu/(1 - nu)$ 为 n 次浮点舍入后计算结果与正确结果的相对误差。书中为避免对常数的讨论，还定义了 $\tilde{\gamma}_n = \gamma_{cn}$ 。本文由于在所有需要使用这个参数的地方都已经将常数 c 显式计算出来，因此不再使用此参数。

对于某个特定的运算序列，我们用

$$|\theta_n| \leq \gamma_n, \quad \theta_n \in \mathbb{R}$$

表示该运算序列中可能引入的误差。下文所有 θ 默认满足此条件。

定义 3.3. 符号函数

我们规定

$$\text{sign}(x) = \begin{cases} 1, & x \geq 0 \\ -1, & x < 0 \end{cases}$$

注意这个 sign 的定义与通常的 sign 在 0 处定义不同。这个函数只是旨在防止在 $x_1 = 1$ 时发生退化导致除 0。

对于计算过程中的任意中间值 a ，我们记 a 为其在精确计算时得到的结果， $\hat{a}$ 为其在有浮点误差的环境中计算得到的结果。

3.2 基础浮点运算的误差估计

本节我们将跟随 Basic.lean 中的顺序，给出一些浮点运算中误差估计的基础结果，并证明一些在证明中会用到的引理。

定理 3.4. 浮点误差阶数的可加性

```
lemma gamma_mul_le (n₁ n₂ : N)
  (hnu : ((n₁ + n₂) : R) * (u : R) < 1) :
  (γ u n₁ + 1) * (γ u n₂ + 1) ≤ γ u (n₁ + n₂) + 1
```

即 n_1 次和 n_2 次的误差叠加后不超过 $n_1 + n_2$ 次的误差。

证明. 展开定义可知结果等价于

$$\frac{1}{1 - un_1} \cdot \frac{1}{1 - un_2} \leq \frac{1}{1 - u(n_1 + n_2)}$$

当分母都为正时成立。 □

定义 3.5. 误差界

```
def diff_gamma_bound (n : N) (x x' : R) : Prop :=
  |x - x'| ≤ γ u n * |x|
def diff_gamma_bound_theta (n : N) (x x' : R) : Prop :=
  ∃ θ : R, x' = x * (1 + θ) ∧ |θ| ≤ γ u n
```

为了定量的控制误差的阶数，我们定义两个等价的误差类表示 x' 在 x 的 γ_n 误差界内。第一个定义直接给出了误差界的定义。第二个定义则使用存在论断给出相同命题，在需要对误差做复杂计算时更方便。两个定义显然是等价的。

引理 3.6. 误差的保号性

```
lemma pos_diff_gamma_bound_pos (n : N) (x x' : R)
  (hnu : 2 * (n : R) * (u : R) < 1) (hx : x > 0)
  (hbound : diff_gamma_bound u n x x') :
  x' > 0
```

$\gamma_n < 1$ 时浮点误差不会导致计算结果的正负号改变。

定理 3.7. 误差的传递性

```
theorem diff_gamma_bound_trans (n1 n2 : N) (x x' x'' : R)
  (hnu : ((n1 + n2) : R) * (u : R) < 1)
  (hbound1 : diff_gamma_bound u n1 x x')
  (hbound2 : diff_gamma_bound u n2 x' x'') :
  diff_gamma_bound u (n1 + n2) x x''
```

定理 3.4 的应用。若 x 经历了一次 n_1 阶误差界的计算得到 x' ，又经历了一次 n_2 阶误差界的计算得到 x'' ，则 x'' 在 x 的 $n_1 + n_2$ 阶误差界内。

误差阶数满足可加性，从而可以推出在复合运算下误差界的传递性。

定理 3.8. 加法运算误差的界

```
lemma add_bound_nonneg
  (nx ny : N) (x y x' y' : R)
  (hx : x ≥ 0) (hy : y ≥ 0)
  (hnu : ((nx ⊔ ny) : R) * (u : R) < 1)
  (hboundx : diff_gamma_bound u nx x x')
  (hboundy : diff_gamma_bound u ny y y') :
  diff_gamma_bound u (nx ⊔ ny) (x + y) (x' + y')
```

当 $x, y \geq 0$ 时，若 $x' = (1 + \theta_{n_x})x$ ， $y' = (1 + \theta_{n_y})y$ ，那么 $x' + y'$ 被 $x + y$ 在 x', y' 中误差阶数

较高的阶数控制。

证明. 由定义可知

$$\begin{aligned}
 |(x' + y') - (x + y)| &\leq |x' - x| + |y' - y| \\
 &\leq \gamma_{nx}|x| + \gamma_{ny}|y| \\
 &\leq \gamma_{\max nx, ny}(|x| + |y|) \\
 &= \gamma_{\max nx, ny}|x + y|
 \end{aligned}$$

□

最后一步中用到了 x, y 同号的性质。在 Lean 代码中将其分成了两种情况。

定理 3.9. 乘法运算误差的界

```

lemma mul_bound
  (nx ny : N) (x y x' y' : R)
  (hnu : ((nx + ny) : R) * (u : R) < 1)
  (hboundx : diff_gamma_bound u nx x x')
  (hboundy : diff_gamma_bound u ny y y') :
  diff_gamma_bound u (nx + ny) (x * y) (x' * y')

```

若 $x' = (1 + \theta_{nx})x$, $y' = (1 + \theta_{ny})y$, 那么 $x'y'$ 被 xy 在 $n_x + n_y$ 阶误差界内控制。

证明. 由定义可知

$$\begin{aligned}
 |x'y' - xy| &\leq |x'y' - x'y| + |x'y - xy| \\
 &\leq |x'||y' - y| + |y||x' - x| \\
 &\leq (1 + \gamma_{nx})|x| \cdot \gamma_{ny}|y| + |y| \cdot \gamma_{nx}|x| \\
 &\leq (\gamma_{nx} + \gamma_{ny} + \gamma_{nx}\gamma_{ny})|x||y| \\
 &\leq \gamma_{nx+ny}|xy|
 \end{aligned}$$

□

定义 3.10. 浮点误差

```

def fl (x x' : R) : Prop :=
  ∃ δ : R, |δ| ≤ (u : R) ∧ x * (1 + δ) = x'

```

对于一次基本的浮点运算，我们定义其结果 x' 与正确结果 x 之间的关系为存在一个 δ 使得 $x' = x(1 + \delta)$ 且 $|\delta| \leq u$ 。也即误差的相对比值被 u 控制。

引理 3.11. 浮点误差界与 γ 误差界的关系

```

lemma fl_to_gamma_bound (x x' : R) (hu : (u : R) < 1) :
  fl u x x' → diff_gamma_bound u 1 x x'

```

显然由于 $u > \gamma_1$ ，一次基本的浮点运算的误差界被 γ_1 控制。

定义 3.12. 向量的浮点误差界

```
def diff_gamma_bound_vec (n : N) {m : N} (x : Vec m) (x' : Vec m) :
  Prop := ∀ i, diff_gamma_bound u n (x i) (x' i)
```

向量误差在 n 阶界内，等价于每一个分量的误差都在 n 阶界内。

定义 3.13. 基础运算的浮点误差界

```
def fl_add (x y z : R) : Prop :=
  fl u (x + y) z
def fl_mul (x y z : R) : Prop :=
  fl u (x * y) z
def fl_smul_vec (a : R)
  (x : Vec n) (y : Vec n) : Prop :=
  ∀ i, fl u (a * x i) (y i)
```

定义了会用到的加法，乘法及向量数乘三种运算。定义即为运算结果在一次浮点误差界以内。

定义 3.14. 带误差的向量点乘

```
def fl_vec_dot : {n : N} → Vec n → Vec n → R → Prop
| 0, _, _, z => z = 0
| n + 1, x, y, z =>
  ∃ p s : R,
    fl_mul u (x 0) (y 0) p ∧
    fl_vec_dot
      (fun i => x (Fin.succ i))
      (fun i => y (Fin.succ i)) s ∧
    fl_add u p s z
```

定义了带误差的向量点乘。我们递归地定义这个过程。当 $n = 0$ 时，点乘结果为 0。当 $n > 0$ 时，我们先计算一个分量的乘积 p ，再递归计算剩余分量的点乘结果 s ，最后将两者相加得到最终结果 z 。每一步运算都满足一次浮点误差界。

注意这里相比于真实场景中的向量内积，额外引入了 $n = 1$ 时一次没有必要的 $x_1 y_1 + 0$ 运算，并且没有额外规定这次运算的误差为 0。这会导致给出的内积运算的界从 n 阶变为 $n + 1$ 阶，对得到的结论影响不大，并且可以简化证明。

定理 3.15. 向量自内积误差的界

```
theorem fl_vec_dot_bound
  (n : N)
  (hnu : ((n : R) + 1) * (u : R) < 1)
  (x : Vec n) (z : R)
  (hfl : fl_vec_dot u x x z) :
  diff_gamma_bound u (n + 1) (x · x) z
```

向量内积自身的计算结果在 $n + 1$ 阶误差界内。与原书中第 3.1 章得到的结论不同，我们将条件修改为向量与自身内积（在下面的证明中也只需要用到这种内积），而不是任意的两个向量

点乘。从而我们可以得到更强的、直接关于计算得数的误差界。

证明. 记 $s_i = x_1x_1 + \cdots + x_ix_i$ 。我们有

$$\begin{aligned}
 \hat{s}_1 &= fl(0 + fl(x_1x_1)) \\
 &= fl(x_1x_1)(1 + \theta_1) \\
 &= x_1x_1(1 + \theta_2) \\
 \hat{s}_i &= fl(\widehat{s_{i-1}} + fl(x_ix_i)) \\
 &= (\widehat{s_{i-1}} + x_ix_i(1 + \theta_1))(1 + \theta_1) \\
 &= (s_{i-1}(1 + \theta_i) + x_ix_i(1 + \theta_1))(1 + \theta_1) \\
 &= (s_{i-1} + x_ix_i)(1 + \theta_{i+1}) \\
 &= s_i(1 + \theta_{i+1})
 \end{aligned}$$

即得 $\hat{s}_n = s_n(1 + \theta_{n+1})$ 。 □

引理 3.16. 误差界受开方运算的影响

```

lemma sqrt_trans_gamma_bound (n : ℕ) (x x' : ℝ) (hx : x ≥ 0)
  (hnu : 2 * (n : ℝ) * (u : ℝ) < 1)
  (h_bound : diff_gamma_bound u n x x') :
  diff_gamma_bound u n (Real.sqrt x) (Real.sqrt x')

```

若 $x' = x(1 + \theta_n)$ 且 $x \geq 0$ ，则 $\sqrt{x'}$ 仍然被 $\sqrt{x}$ 在 n 阶误差界内控制。这里其实可以放成 $n/2$ 阶，但为了简化证明我们直接放成 n 阶。

引理 3.17. 误差界受取倒数运算的影响

```

lemma div_inv_trans_gamma_bound (n : ℕ) (x x' : ℝ) (hx : x > 0)
  (hnu : 2 * (n : ℝ) * (u : ℝ) < 1)
  (h_bound : diff_gamma_bound u n x x') :
  diff_gamma_bound u (2 * n) (1 / x) (1 / x')

```

若 $x' = x(1 + \theta_n)$ 且 $x > 0$ ，则 $1/x'$ 被 $1/x$ 在 $2n$ 阶误差界内控制。

原书中基于 u 很小的假设，直接得出了 $1/(1 + \theta_n) = 1 + \theta_n$ 。在更严格的 Lean 中，我们无法保证 $1/(1 - \gamma_n) \leq 1 + \gamma_n$ 。因此我们被迫把界放宽到 $2n$ 阶。

证明. 只需证明

$$\begin{cases} 1/(1 - \gamma_n) \leq 1 + \gamma_{2n} \\ 1/(1 + \gamma_n) \geq 1 - \gamma_{2n} \end{cases}$$

代入 γ ，上两式即为

$$\begin{cases} 1/(1 - \frac{nu}{1-nu}) \leq 1 + \frac{2nu}{1-2nu} \\ 1/(1 + \frac{nu}{1-nu}) \geq 1 - \frac{2nu}{1-2nu} \end{cases}$$

由假设 $2nu < 1$ ，上两式的分母都为正。通分得到

$$\begin{cases} (1 - \frac{nu}{1-nu}) \cdot (1 + \frac{2nu}{1-2nu}) \geq 1 & (1) \\ (1 + \frac{nu}{1-nu}) \cdot (1 - \frac{2nu}{1-2nu}) \leq 1 & (2) \end{cases}$$

我们分开证明这两个不等式。

对式 (1):

$$\begin{aligned} (1 - \frac{nu}{1-nu}) \cdot (1 + \frac{2nu}{1-2nu}) &= 1 + \gamma_{2n} - \gamma_n - \gamma_n \gamma_{2n} \\ &\geq 1 + \gamma_{2n} - 2\gamma_n \\ &= 1 + \frac{2nu}{1-2nu} - \frac{2nu}{1-nu} \\ &\geq 1 \end{aligned}$$

对式 (2):

$$\begin{aligned} (1 + \frac{nu}{1-nu}) \cdot (1 - \frac{2nu}{1-2nu}) &= 1 - \gamma_{2n} + \gamma_n - \gamma_n \gamma_{2n} \\ &= 1 + (\gamma_n - \gamma_{2n}) - \gamma_n \gamma_{2n} \\ &\leq 1 \end{aligned}$$

□

以上为 `Basic.lean` 中证明的和定义的全部重要内容。我们在后续的证明中会使用这些结果。

3.3 Householder 过程

定义 3.18. Householder 算法

$\forall x \in \mathbb{R}^n, x \neq 0$ ，使用如下的 Householder 算法来求解 $(I - vv^T)x = -(\text{sign } x_1) \|x\|_2 e_1$:

$$\begin{cases} l' = x^T x \\ l = \sqrt{l'} \\ s = \text{sign}(x) \cdot l \\ v_1 = x_1 + s \\ p = sv_1 \\ \beta = 1/p \\ b = \sqrt{\beta} \\ v' = \begin{cases} v_1, & i = 1 \\ x_i, & i > 1 \end{cases} \\ v = bv' \end{cases}$$

```
structure Householder (n : N) (hn : n ≠ 0) where
```



```

l' : R
l  : R
s  : R
v0 : R
v' : Vec n
p  : R
β  : R
b  : R
v  : Vec n

def householder (n : N) (x : Vec n) (hn : n ≠ 0) :
  Householder n hn := by
    let i₀ : Fin n := ⟨0, Nat.pos_of_ne_zero hn⟩
    let l' := x · x
    let l  := Real.sqrt l'
    let s  := sign x hn * l
    let v0 := x i₀ + s
    let p  := s * v0
    let β  := 1 / p
    let b  := Real.sqrt β
    let v' : Vec n := fun i => if i = i₀ then v0 else x i
    let v  := b · v'
    exact {l' := l', l := l, s := s, v0 := v0,
           v' := v', p := p, β := β, b := b, v := v
          }

```

定理 3.19. Householder 算法的正确性

```

theorem e₁_of_householder
  (n : N)
  (x : Vec n)
  (hn : n ≠ 0)
  (hx : x ≠ 0) :
    let hv := householder n x hn
    let P  := 1 - (vecMulVec hv.v hv.v)
    P · x = e₁ n x hn

```

证明. 反复代入可知

$$v' = x - (- (\text{sign } x_1) \|x\|_2 e_1).$$

由第 2.1.2 节的结论我们有

$$(I - \frac{2}{v'^T v'} v' v'^T) x = -(\text{sign } x_1) \|x\|_2 e_1.$$

另一方面, 由定义 $p = s v_1 = s x_1 + s^2$, 并且

$$v'^T v' = x^T x + 2s x_1 + s^2 = 2s x_1 + 2s^2 = 2p.$$

因此

$$b = \sqrt{\beta} = \sqrt{\frac{1}{p}} = \sqrt{\frac{2}{v'^T v'}}.$$

令 $v = b v'$, 则有

$$v^T v = b^2 v'^T v' = \frac{2}{v'^T v'} v'^T v' = 2.$$

从而

$$I - \frac{2}{v'^T v'} v' v'^T = I - v v^T.$$

因此最后一步中的缩放因子 b 将向量归一化, 使 Householder 矩阵从标准形式 $I - \frac{2}{v'^T v'} v' v'^T$ 化为计算上更为简洁的 $I - v v^T$. $\square$

以上的证明过程中引用了前序的结论, 然而在 Lean 中我们需要单独证明。详细的证明过程可以查看代码库中的 `Householder.lean`。

3.4 Householder 法的误差估计

定义 3.20. 带误差的 Householder 过程

```
def fl_householder (n : ℕ) (x : Vec n) (hn : n ≠ 0)
  (hv_fl : Householder n hn) : Prop :=
  -- (1) l' = fl(xTx)
  fl_vec_dot u x x hv_fl.l' ∧
  -- (2) l = fl(sqrt(l'))
  fl u (Real.sqrt hv_fl.l') hv_fl.l ∧
  hv_fl.s = sign x hn * hv_fl.l ∧
  -- (3) v_o = fl(x_o + s)
  fl_add u (x <0, Nat.pos_of_ne_zero hn) hv_fl.s hv_fl.v0 ∧
  -- (4) p = fl(s * v_o)
  fl_mul u hv_fl.s hv_fl.v0 hv_fl.p ∧
  -- (5) β = fl(1 / p)
  fl u (1 / hv_fl.p) hv_fl.β ∧
  -- (6) b = fl(√β)
  fl u (Real.sqrt hv_fl.β) hv_fl.b ∧
  -- (7) v = fl(b * v')
  hv_fl.v' = (fun i => if i = <0, Nat.pos_of_ne_zero hn>
```

```

      then hv_fl.v0
      else (x i)) ^
    fl_smul_vec u hv_fl.b hv_fl.v' hv_fl.v

```

我们将带误差的 **Householder** 过程定义为若干个变量之间的链式关系，每个变量都和之前的变量满足某个浮点误差约束。我们将证明每传递一段，误差阶数都会增加一些，最终得到结果的误差阶数。

定理 3.21. Householder 算法误差的界（即参考文献中的引理 19.1）

```

theorem Lemma_19_1 (n : ℕ) (x : Vec n)
  (hn : n ≠ 0) (hx : x ≠ 0)
  (hv_fl : Householder n hn)
  (hnu : (8 * n + 26 : ℝ) * (u : ℝ) < 1)
  (hfl : fl_householder u n x hn hv_fl) :
    diff_gamma_bound_vec u (5 * n + 18)
    (householder n x hn).v hv_fl.v

```

即 $\forall 1 \leq i \leq n, \exists \delta_i, |\delta_i| \leq \gamma_{5n+18}, \hat{v}_i = v_i(1 + \theta_i)$ 。

证明. 我们从前向后，逐个证明计算每个变量的误差的界。首先我们假设精确的 **Householder** 过程为 **hv**:

```

let hv := householder n x hn
  直接应用  $l'$  的定义和定理 3.15，我们有

```

```

have l'_bound : diff_gamma_bound u (n + 1) hv.l' hv_fl.l'

```

由引理 3.16，上面的界可以传递到 $\sqrt{l'}$ 。再考虑一次开根运算引入的误差，我们有：

```

have l_bound : diff_gamma_bound u (n + 2) hv.l hv_fl.l

```

乘 1 或 -1 不会引入误差，因此 s 的界与 l 相同：

```

have s_bound : diff_gamma_bound u (n + 2) hv.s hv_fl.s

```

由 s 的定义，我们知道 s 与 x_1 同时非负或非正。因此由引理 3.8 以及它的非正版本，上面的界可以传递到 $\hat{x}_1 + \hat{s}$ 。再考虑一次加法引入的误差，我们有：

```

have v0_bound : diff_gamma_bound u (n + 3) hv.v0 hv_fl.v0

```

由上文证明， $\hat{s}$ 有 $n+2$ 阶界， $\hat{v}_0$ 有 $n+3$ 阶界，由定理 3.9， $\hat{s} \cdot \hat{v}_0$ 有 $2n+5$ 阶界。再考虑一次乘法引入的误差，我们有：

```

have p_bound : diff_gamma_bound u (2 * n + 6) hv.p hv_fl.p

```

由引理 3.17，可以得到 $\frac{\hat{p}}{p}$ 的界：

```

have p_bound' : diff_gamma_bound u (4 * n + 12) (1 / hv.p) (1 / hv_fl.p)

```

$\hat{\beta} = fl(\frac{\hat{p}}{p})$ ，因此相比上面再增加一次误差：

```

have beta_bound : diff_gamma_bound u (4 * n + 13) hv.beta hv_fl.beta

```

和 l 的界相同，开方不会放大输入的误差，再考虑一次开根运算引入的误差，我们有：

```

have b_bound : diff_gamma_bound u (4 * n + 14) hv.b hv_fl.b

```

最后，对 x_1 以外的分量， $\hat{v}_i = fl(\hat{b} \cdot x_i)$ ，由定理 3.9， $\hat{v}_i$ 的界为 $4n+15$ 阶。对于第一个分量， $\hat{v}_0 = \hat{b} \cdot \hat{v}_0$ ，同样由定理 3.9， $\hat{v}_0$ 的界为 $5n+18$ 阶。因此，向量每个分量的界都被 $5n+18$

阶控制。



参考文献

- [1] Nicholas G. Higham (2002). *Accuracy and Stability of Numerical Algorithms (Second Edition)*, 61-76, 353-362. Society for Industrial and Applied Mathematics.
- [2] Guoxiong Gao, Haocheng Ju, Jiedong Jiang, Zihan Qin, Bin Dong (2024). A Semantic Search Engine for Mathlib4. *EMNLP 2024 findings*. <https://arxiv.org/abs/2403.13310>.
- [3] Lean and Mathlib4 Maintainer team. Documentation page for Mathlib4 and Lean, https://leanprover-community.github.io/mathlib4_docs

附录

致谢

北京大学学位论文原创性声明和使用授权说明

原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品或成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律结果由本人承担。

论文作者签名：

日期： 年 月 日

学位论文使用授权说明

本人完全了解北京大学关于收集、保存、使用学位论文的规定，即：

- 按照学校要求提交学位论文的印刷本和电子版本；
- 学校有权保存学位论文的印刷本和电子版，并提供目录检索与阅览服务，在校园网上提供服务；
- 学校可以采用影印、缩印、数字化或其它复制手段保存论文；

论文作者签名：

导师签名：

日期： 年 月 日